A3 Delta Lake

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A3 Delta Lake 3
A3-L1 Idempotent MERGE, and why you deduplicate the source first 3
A3-L2 Schema enforcement is on by default. Evolve on purpose. 9
A3-L3 Time travel, RESTORE, OPTIMIZE and VACUUM 14
Track A best-practice checklist 19
Appendix: solutions and answer keys 20

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A3 Delta Lake

Roadmap: Months 16 to 18: MLOps and deployment; Current work. MERGE, idempotent writes, schema enforcement and evolution, time travel, maintenance.

A3-L1 Idempotent MERGE, and why you deduplicate the source first

Concept

A pipeline is **idempotent** when running it twice with the same input leaves the table in the same state as running it once. Retries, reruns and redeliveries happen every week in production, so idempotence is not optional.

An append is not idempotent. Run `write.mode("append")` twice and every row lands twice. A Delta `MERGE` is idempotent when two conditions hold:

1. The `ON` condition covers every column of the table's grain. For a silver table with one row per respondent per wave, that means `respondent_id` **and** `wave_id`.
2. The source holds at most one row per key.

The second condition catches people out. When two source rows match the same target row, Delta cannot tell which one should win. It stops the `MERGE` with the error `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. When neither row matches a target row, both rows are inserted, and your grain is broken without any error.

So deduplicate the source before you merge. Rank the rows for each key by a version column, such as `delivery_version` or `ingestion_timestamp`, and keep the newest. Use a window with `row_number()`. Do not use `dropDuplicates()`: it keeps an arbitrary row per key, so a rerun keeps a different version with no warning.

The pattern for every silver load:

- Keep the newest source row per key.
- `MERGE` on the full grain.
- `whenMatchedUpdateAll()` for redelivered rows, `whenNotMatchedInsertAll()` for new ones.

Bronze stays append-only, because bronze records every delivery as received. Silver is where you resolve versions.

Why it matters

A supplier redelivers a wave with corrected weights. The file repeats 37 respondents, because the supplier's export joined two tables. The silver job uses the reference `upsert_silver`, which merges without deduplicating. The `MERGE` fails at 02:00 with `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Gold does not refresh, and the morning Power BI report shows last week's numbers.

The on-call engineer "fixes" it by switching the job to append. The next run succeeds. Silver now holds the wave twice. Reach percentages look normal, because both the numerator and the denominator doubled. Weighted population estimates are twice their true size for a month, until a reconciliation check against the weight total catches it.

Deduplicating on the full grain before the `MERGE` prevents both failures.

Worked example: weak code

```
1  from delta.tables import DeltaTable
2  from pyspark.sql import DataFrame, SparkSession
3
1  spark = SparkSession.builder.getOrCreate()
5
6
2  def upsert_silver(new_df: DataFrame, target_table: str, merge_keys: list)
       -> None:
8      """Merge new data into silver table."""
9      delta_table = DeltaTable.forName(spark, target_table)
10     merge_condition = " AND ".join(
11         [f"target.{k} = source.{k}" for k in merge_keys]
12     )
13     (
14         delta_table.alias("target")
3          .merge(new_df.alias("source"), merge_condition)
16         .whenMatchedUpdateAll()
17         .whenNotMatchedInsertAll()
18         .execute()
19     )
```

1. **Line 4.** A session created at import time. Tests cannot inject a local session, and importing the module has a side effect.
2. **Line 7.** `list` with no element type. mypy cannot check that callers pass column names.

3. **Line 15.** The source goes straight into the MERGE. One repeated key in a redelivery fails the whole job.

Worked example: strong code

```python
from delta.tables import DeltaTable
from pyspark.sql import DataFrame, SparkSession, Window
from pyspark.sql import functions as F


def latest_per_key(df: DataFrame, keys: list[str], order_col: str) -> DataFrame:
    """Keep the newest row for each key.

    Args:
        df: Source rows, possibly with repeated keys.
        keys: Columns that identify one target row.
        order_col: Column that ranks versions. Highest wins.

    Returns:
        One row per key.
    """
    window = Window.partitionBy(*keys).orderBy(F.col(order_col).desc())
    return (
        df.withColumn("_rank", F.row_number().over(window))
        .filter(F.col("_rank") == 1)
        .drop("_rank")
    )


def upsert_silver(
    spark: SparkSession,
    source: DataFrame,
    target_table: str,
    keys: list[str],
    order_col: str,
) -> None:
    """MERGE the newest source row per key into a silver table.

    Running it twice with the same source leaves the table unchanged.
    """
    condition = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (
        DeltaTable.forName(spark, target_table).alias("t")
        .merge(latest_per_key(source, keys, order_col).alias("s"), condition)
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
```

```
43      )
```

1. **Line 3.** Import the module as `F`. Importing `sum` or `max` by name shadows the Python built-ins.
2. **Line 17.** One partition per key, newest version first. The result is the same on every run.
3. **Line 19.** `row_number()` gives exactly one row rank 1 per key, even when versions tie. `rank()` would keep ties.
4. **Line 26.** The session is a parameter. A test passes a local session, a job passes the cluster session.
5. **Line 39.** Deduplicate inside the MERGE call, so no caller can skip it.

The strong version is idempotent for any source: repeated keys, a retry or a full redelivery. The weak version works only while the supplier never repeats a row.

Recap card

- **When is a pipeline idempotent?** When running it twice with the same input leaves the table as one run would.
- **Two conditions for an idempotent Delta MERGE** The ON condition covers the full grain, and the source has at most one row per key.
- **What happens when two source rows match one target row?** The MERGE fails with DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE.
- **Why not dropDuplicates() before a MERGE?** It keeps an arbitrary row per key. Use row_number() over a window ordered by version, newest first.

Practice

Exercise 1. Multiple choice (level 1)

The silver table holds one row for respondent `R1` in wave `W1`. A redelivery holds **two** rows for `R1` in `W1`. You run a `MERGE` on `respondent_id` and `wave_id` with `whenMatchedUpdateAll()` and `whenNotMatchedInsertAll()`. What happens?

- a) The target row takes the values of the last source row.
- b) The MERGE fails with an error.
- c) Both source rows are inserted as new rows.
- d) The target row takes the values of the first source row.

Why?

- r1) Two source rows match one target row, and Delta cannot tell which should win.
- r2) Delta enforces a primary key on the target table.
- r3) The source DataFrame must be cached before a MERGE.

Exercise 2. Fill the gap (level 2)

Complete the chain so that redelivered rows update, new rows insert, and the operation runs.

```
(
    DeltaTable.forName(spark, "research_silver.nat_respondents").alias("t")
    .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id")
    ._____(1)()
    ._____(2)()
    ._____(3)()
)
```

Exercise 3. Predict the output (level 2)

`silver` starts empty. `source` holds 3 rows with 3 distinct `(respondent_id, wave_id)` keys. `load()` calls the strong `upsert_silver` from the worked example. What does the last line print?

```
load(source)  # MERGE on respondent_id and wave_id
load(source)  # the scheduler retries the same run
print(spark.table("silver").count())
```

Exercise 4. Spot the bug (level 3)

Silver has one row per respondent per wave. After wave `W2` loads, every wave `W1` row has disappeared. Tap the line with the bug, then fix the code.

```
def upsert_respondents(spark: SparkSession, source: DataFrame) -> None:
    """MERGE one wave of respondents into silver. Grain: respondent per wave."""
    latest = latest_per_key(source, ["respondent_id", "wave_id"], "delivery_version")
    (
        DeltaTable.forName(spark, "research_silver.nat_respondents").alias("t")
        .merge(latest.alias("s"), "t.respondent_id = s.respondent_id")
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 4)

Write `latest_per_key(df, keys, order_col)`. It returns one row per key: the row with the highest `order_col`. Use type hints and a docstring. The checker reads your code without running Spark.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Keep the row with the highest order_col for each key.
def latest_per_key(
    df: DataFrame,
    keys: list[str],
    order_col: str,
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import DataFrame, Window
from pyspark.sql import functions as F


def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
   DataFrame:
    ...
```

Exercise 6. Put in order (level 5)

Put the steps of an idempotent silver load in order.

- Write one record to the pipeline run log
- Rename and cast columns from the Variable Registry
- Validate the bronze delivery against its data contract
- MERGE into silver on respondent_id and wave_id
- Run the table health check on silver
- Keep the newest row per respondent and wave

Practical: Prove a MERGE is idempotent

Platform: Databricks Free Edition. Run this in Databricks Free Edition on serverless compute. It creates a schema called `learning_a3` in your current catalog, loads the same data twice, and shows that the row count does not change. It then repeats a key and shows the error.

Download the notebook `A3-L1.ipynb` from the app.

Sources

- MERGE fails with DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE

when several source rows match one target row. (verified: practice/tests/test_a3_l1_merge.py reproduces the error on delta-spark 4.0.1. Docs: https://docs.databricks.com/aws/en/sql/language-manual/delta-merge-into)

- DeltaTable.forName, alias, merge, whenMatchedUpdateAll, whenNotMatchedInsertAll, whenNotMatchedBySourceDelete and execute exist in delta-spark 4.0. (verified: practice/tests/test_a3_l1_merge.py checks the methods on delta-spark 4.0.1.)
- Free Edition users can create schemas in the workspace catalog, whose name matches the workspace. (search-only: https://docs.databricks.com/aws/en/catalogs/default)

A3-L2 Schema enforcement is on by default. Evolve on purpose.

Concept

A Delta table has a schema, and writes must match it. This is **schema enforcement**, and it is on by default. Tested on delta-spark 4.0.1:

Write	Result
Append with an extra column	Fails: schema mismatch
Append with a column missing	Succeeds: the missing column is NULL
Append with a clashing type (INT into DOUBLE)	Fails: `DELTA_FAILED_TO_MERGE_FIELDS`
Overwrite with a different schema	Fails unless `overwriteSchema` is true
MERGE with `updateAll`/`insertAll` and an extra source column	Succeeds, and **silently drops the extra column**

Enforcement is your friend. A failing write is a cheap, early signal that a delivery changed.

Schema evolution changes the table's schema to accept the new shape. Delta offers it per write:

- Append: `.option("mergeSchema", "true")` adds new columns.
- MERGE: `.withSchemaEvolution()` on the merge builder adds new source columns.
- Planned change: `ALTER TABLE ... ADD COLUMN (region STRING)` in a reviewed migration.

Evolve on purpose, never by default. Two habits cause trouble:

- Leaving `mergeSchema` on every silver write. A supplier typo such as `final_wieght` becomes a new, permanent column, and the real weight is NULL for that wave.
- Setting `spark.databricks.delta.schema.autoMerge.enabled` for the session, which Databricks now labels legacy. It turns evolution on for every MERGE in the job, including ones you did not think about.

The safe pattern: validate the incoming columns against the registry first (lesson A2-L2). If a column is new and expected, add it with `ALTER TABLE` or evolve that one write. If it is unexpected, fail the run and ask the supplier.

Note the silent MERGE case in the table. Enforcement does not protect you there. Your column check does.

Why it matters

A silver job appends with `mergeSchema` always on, "to avoid schema errors". A supplier's export misspells `final_weight` as `final_wieght` for one wave. The append succeeds. Silver gains a permanent `final_wieght` column, and `final_weight` is NULL for 2,000 respondents. Gold sums NULL weights to a smaller base, and weighted reach for that wave is computed on the wrong population.

Without `mergeSchema`, the append fails with a schema mismatch naming the unexpected column. The fix takes minutes, and silver never holds the bad wave.

Worked example: weak code

```
1  spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
2
3
4  def append_wave(df, table):
2      df.write.format("delta").mode("append").option("mergeSchema",
       "true").saveAsTable(table)
```

1. **Line 1.** Turns evolution on for every MERGE in the session, not just the one you meant.
2. **Line 5.** Every append may now add columns. A typo in a delivery becomes part of the table.

Worked example: strong code

```
1  def check_columns_match(df: DataFrame, expected: list[str]) -> None:
2      """Fail when the delivery has unexpected or missing canonical
       columns."""
```

```
    unexpected = sorted(set(df.columns) - set(expected))
    missing = sorted(set(expected) - set(df.columns))
    if unexpected or missing:
(1)     raise ValueError(f"schema drift: unexpected={unexpected} missing={missing}")


def append_wave(df: DataFrame, table: str, expected: list[str]) -> None:
    """Append after an explicit column check. Delta enforcement stays on."""
    check_columns_match(df, expected)
(2)  df.write.format("delta").mode("append").saveAsTable(table)


def add_planned_column(spark: SparkSession, table: str, column: str, sql_type: str) -> None:
    """Planned schema change, run from a reviewed migration."""
(3)  spark.sql(f"ALTER TABLE {table} ADD COLUMN ({column} {sql_type})")
```

1. **Line 6.** Both directions checked. A missing column would otherwise land as NULL without an error.
2. **Line 12.** No mergeSchema. An unexpected shape fails loudly.
3. **Line 17.** New columns arrive through a reviewed change, not as a side effect of a write.

Schema changes become decisions a person reviews, not accidents a delivery causes.

Recap card

- **Delta append with an extra column, no options** Fails with a schema mismatch. Enforcement is on by default.
- **MERGE with updateAll and an extra source column, no evolution** Succeeds and silently ignores the extra column. Only your own column check catches it.
- **Evolve one write on purpose** Append: option mergeSchema. MERGE: .withSchemaEvolution(). Planned: ALTER TABLE ADD COLUMN.
- **Why not set autoMerge.enabled for the session?** It turns evolution on for every MERGE in the job, including ones you did not intend.

Practice

Exercise 1. Multiple choice (level 1)

Silver has columns `respondent_id`, `wave_id`, `final_weight`. You append a DataFrame with those three columns plus `region`, without any options. What happens on delta-spark 4.0?

- a) The append succeeds and region is added.
- b) The append succeeds and region is dropped.
- c) The append fails with a schema mismatch.
- d) Region is stored in a hidden rescue column.

Why?

- r1) Schema enforcement rejects writes with columns the table does not have.
- r2) Delta tables cannot change schema after creation.

Exercise 2. Predict the output (level 2)

Silver `s` has columns `id` and `w`. The source has `id`, `w` and `region`. What does the last line print on delta-spark 4.0?

```
(
    DeltaTable.forName(spark, "s").alias("t")
    .merge(source.alias("src"), "t.id = src.id")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)
print(spark.table("s").columns)
```

Exercise 3. Fill the gap (level 3)

You reviewed the supplier's new `region` column and want this one MERGE to add it to silver. Complete the builder.

```
(
    DeltaTable.forName(spark, table).alias("t")
    .merge(source.alias("s"), condition)
    ._____(1)()
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)
```

Exercise 4. Write the function (level 4)

Write `schema_drift(actual, expected)`. It takes two lists of column names and returns a dict with sorted lists under `"unexpected"` (in actual, not expected) and `"missing"` (in expected, not actual). Then write `assert_no_drift(actual, expected)` that raises `ValueError` naming both lists when either is non-empty.

Your code must pass these checks:

- Same columns in a different order is no drift.
- Report both directions: the typo is unexpected, and the real column is missing.
- Return sorted lists, so messages and tests are stable.
- `assert_no_drift` must raise ValueError that names the drifted columns.
- No drift must return None without raising.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Compare delivered columns with the registry.
def schema_drift(
    actual: list[str],
    expected: list[str],
) -> dict[str, list[str]]: ...

# Raise ValueError when the delivery's columns differ from the registry.
def assert_no_drift(
    actual: list[str],
    expected: list[str],
) -> None: ...
```

Starter code:

```python
def schema_drift(actual: list[str], expected: list[str]) -> dict[str, list[str]]:
    ...


def assert_no_drift(actual: list[str], expected: list[str]) -> None:
    ...
```

Exercise 5. Spot the bug (level 5)

This silver job evolved the schema on every run for a month, and silver now has 14 columns nobody asked for. Tap the line that causes it, then fix the function.

```python
def upsert_silver(spark: SparkSession, source: DataFrame, table: str, keys: list[str]) -> None:
    """MERGE a deduplicated source into silver."""
```

```
    spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled",
"true")
    condition = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (
        DeltaTable.forName(spark, table).alias("t")
        .merge(source.alias("s"), condition)
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )
```

Write down the line number of the bug. Then write the fix.

Sources

- Append with an extra column fails; with a missing column fills NULL; with a type clash fails with DELTA_FAILED_TO_MERGE_FIELDS; overwrite with a new schema needs overwriteSchema. (verified: Run on delta-spark 4.0.1 in this repository.)
- MERGE with updateAll/insertAll ignores extra source columns without evolution; DeltaMergeBuilder.withSchemaEvolution() adds them. (verified: Run on delta-spark 4.0.1 in this repository.)
- spark.databricks.delta.schema.autoMerge.enabled turns on schema evolution for MERGE across the session. (verified: https://docs.databricks.com/aws/en/data-engineering/schema-evolution (checked 2026-09-25))

A3-L3 Time travel, RESTORE, OPTIMIZE and VACUUM

Concept

Every Delta write creates a new table **version**. The transaction log keeps the list, and you can read any version still on storage.

- `DESCRIBE HISTORY t` lists versions with the operation, user and metrics.
- `spark.read.option("versionAsOf", 12).table("t")` or `SELECT * FROM t VERSION AS OF 12` reads an old version. `timestampAsOf` and `TIMESTAMP AS OF` work by time.
- `RESTORE TABLE t TO VERSION AS OF 12` makes version 12 current again, as a new version. History is kept, so you can undo the restore.

Time travel is your first response to a bad write: compare the versions, restore, then fix the job.

Two maintenance commands keep tables fast and storage bounded.

OPTIMIZE compacts many small files into fewer large ones. Frequent small appends leave thousands of tiny files, and every query pays to open them. `OPTIMIZE t ZORDER BY (wave_id)` also co-locates rows by a column. On Unity Catalog managed tables, Databricks recommends liquid clustering (`CLUSTER BY`, or `CLUSTER BY AUTO`) with predictive optimization, which runs OPTIMIZE and VACUUM for you.

VACUUM deletes data files that no current version needs and that are older than the retention window. The window comes from `delta.deletedFileRetentionDuration`, which defaults to 7 days. Delta blocks `VACUUM t RETAIN 0 HOURS` with a safety check, because a running reader or writer might still need those files.

VACUUM limits time travel. After it runs, versions whose files it removed can no longer be read. The log itself keeps entries for `delta.logRetentionDuration`, 30 days by default. From Databricks Runtime 18.0, time travel beyond `deletedFileRetentionDuration` is blocked outright. If you need 30 days of recoverable history, set both properties to at least 30 days.

Why it matters

A developer runs a gold rebuild against the wrong wave filter and overwrites three weeks of metrics. The team wants to restore, but a clean-up job ran `VACUUM ... RETAIN 0 HOURS` nightly, with the retention check switched off "to save storage". The overwritten files are gone. Three weeks of gold are rebuilt from silver overnight, and two client extracts go out late.

With the default 7-day retention, `RESTORE TABLE research_gold.nat_metrics TO VERSION AS OF` the last good version takes seconds.

Worked example: weak code

```
❶ SET spark.databricks.delta.retentionDurationCheck.enabled = false;
❷ VACUUM research_gold.nat_metrics RETAIN 0 HOURS;
  3 -- gold was overwritten by mistake last night; rebuild everything
❸ DROP TABLE research_gold.nat_metrics;
```

1. **Line 1.** Switches off the safety check that exists to stop the next line.
2. **Line 2.** Deletes every unreferenced file now. Time travel and RESTORE no longer work.
3. **Line 4.** With history intact, a RESTORE takes seconds. Dropping and rebuilding takes hours.

Worked example: strong code

```
1  -- 1. Find the last good version.
2  DESCRIBE HISTORY research_gold.nat_metrics;
3
4  -- 2. Compare before you act.
5  SELECT wave_id, SUM(weighted_base) AS base
6  FROM research_gold.nat_metrics VERSION AS OF 41
7  GROUP BY wave_id;
8
9  -- 3. Restore. This writes a new version, so it can be undone.
10 RESTORE TABLE research_gold.nat_metrics TO VERSION AS OF 41;
11
12 -- 4. Keep 30 days recoverable.
13 ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES (
14   'delta.deletedFileRetentionDuration' = 'interval 30 days',
15   'delta.logRetentionDuration' = 'interval 30 days'
16 );
```

1. **Line 2.** Each version with its operation, user and metrics such as numOutputRows.
2. **Line 6.** Read the old version side by side with the current one before restoring.
3. **Line 10.** Restore adds a version. The bad write stays in history for the post-mortem.
4. **Line 14.** VACUUM keeps unreferenced files for 30 days, so 30 days of versions stay readable.

History turns a data incident into a two-minute restore. Keep it long enough to be useful.

Recap card

- **Read an old Delta version** VERSION AS OF n or TIMESTAMP AS OF ts in SQL; option versionAsOf in PySpark.
- **What does RESTORE do to history?** Adds a new version with the old content. Nothing is deleted.
- **Default retention windows** Deleted files 7 days (delta.deletedFileRetentionDuration). Log 30 days (delta.logRetentionDuration).
- **Why is VACUUM ... RETAIN 0 HOURS dangerous?** It deletes the files that time travel and RESTORE need, and it breaks readers still using them.
- **Databricks' recommendation for layout and maintenance on UC managed tables** Liquid clustering (CLUSTER BY or CLUSTER BY AUTO) with predictive optimization.

Practice

Exercise 1. Multiple choice (level 1)

A job overwrote gold with bad data one hour ago. VACUUM has not run since. What is the fastest safe recovery?

- a) Drop the table and rebuild it from silver.
- b) `RESTORE TABLE` to the version before the bad write.
- c) Run `VACUUM` to remove the bad files.
- d) Run `OPTIMIZE` to rewrite the table.

Why?

- r1) The earlier version's files are still on storage, and RESTORE makes that version current again.
- r2) RESTORE deletes the bad version from history.

Exercise 2. Fill the gap (level 2)

Read version 41 of the gold table, then make it current.

```
SELECT * FROM research_gold.nat_metrics _____(1) 41;
_____(2) TABLE research_gold.nat_metrics TO VERSION AS OF 41;
```

Exercise 3. Predict the output (level 3)

A new Delta table gets these statements in order. What version number does the RESTORE create?

What you need to know.

`CREATE TABLE t (id INT) USING DELTA` creates a table stored in the Delta format, so it has a transaction log and versions. On Databricks, Delta is the default, and `USING DELTA` states it explicitly.

Each statement that changes the table commits one new version: the `CREATE` is version 0, and every `INSERT`, `UPDATE`, `DELETE` or `MERGE` after it adds one.

```
CREATE TABLE t (id INT) USING DELTA;          -- version 0
INSERT INTO t VALUES (1);                     -- version 1
INSERT INTO t VALUES (2);                     -- version 2
DELETE FROM t WHERE id = 1;                   -- version 3
RESTORE TABLE t TO VERSION AS OF 1;           -- version ?
```

Exercise 4. Spot the bug (level 4)

A nightly maintenance script. After an incident, the team cannot time travel even one day back. Tap the line that causes it, then fix the script.

What you need to know.

`SET key = value` changes a Spark setting for the rest of the session.

`ANALYZE TABLE t COMPUTE STATISTICS` collects table statistics, such as the row count and size in bytes, so the query optimiser picks better plans. It reads data and deletes nothing.

```
OPTIMIZE research_silver.nat_respondents ZORDER BY (wave_id);
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM research_silver.nat_respondents RETAIN 0 HOURS;
ANALYZE TABLE research_silver.nat_respondents COMPUTE STATISTICS;
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

The data governance policy says: "Any silver or gold change must be recoverable for 30 days." Choose the setting for each item.

Options: interval 7 days (default), interval 30 days, interval 0 hours.

1. delta.deletedFileRetentionDuration
2. delta.logRetentionDuration
3. Nightly VACUUM command

Sources

- DESCRIBE HISTORY, VERSION AS OF, versionAsOf, RESTORE TABLE and OPTIMIZE ... ZORDER BY work; RESTORE creates a new version; VACUUM RETAIN 0 HOURS fails the safety check. (verified: Run on delta-spark 4.0.1 in this repository.)
- deletedFileRetentionDuration defaults to 7 days and logRetentionDuration to 30 days; from Databricks Runtime 18.0 time travel beyond deletedFileRetentionDuration is blocked. (verified: https://docs.databricks.com/aws/en/delta/vacuum and https://docs.databricks.com/aws/en/tables/history (checked 2026-09-25))
- Databricks recommends liquid clustering and predictive optimization for Unity Catalog managed tables. (verified: https://docs.databricks.com/aws/en/tables/clustering (checked 2026-09-25))

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A3-L1 Idempotent MERGE, and why you deduplicate the source first

A3-L1 exercise 1

Hints. (1) Count how many source rows match the one target row. (2) Ask yourself which of the two source rows Delta should use for the update. (3) Delta raises DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE in this case.

Answer. b) The MERGE fails with an error. Because: r1) Two source rows match one target row, and Delta cannot tell which should win.

Delta requires at most one source row per matched target row. With two, the update is ambiguous, so the `MERGE` fails with `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Deduplicate the source first.

Common wrong answers.

- `a`: Delta has no notion of 'last' in an unordered DataFrame. Row order in Spark is not guaranteed, so Delta refuses to guess.
- `d`: Spark gives no row order without an explicit sort, so 'first' has no meaning. Delta refuses to guess.
- `c`: Both rows match the existing target row, so neither goes to the insert clause. Duplicate inserts happen only when no target row matches.
- `b|r2`: Right outcome, wrong reason. Delta does not enforce primary keys. The error comes from the ambiguous match, not from a key constraint.
- `b|r3`: Right outcome, wrong reason. Caching has nothing to do with it. The error comes from two source rows matching one target row.

A3-L1 exercise 2

Hints. (1) Two clauses and one action. (2) The clause names start with `whenMatched` and `whenNotMatched`, and both copy every column. (3) The copy-every-column versions end in `All`. The action that runs a MERGE is not `collect`.

Answer.

```
(
    DeltaTable.forName(spark,
"research_silver.nat_respondents").alias("t")
    .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND
t.wave_id = s.wave_id")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)
```

`whenMatchedUpdateAll()` copies every source column onto the matched target row. `whenNotMatchedInsertAll()` inserts rows with no match. `execute()` runs the MERGE.

Common wrong answers.

- g3 = `collect`: `collect()` is a DataFrame action. A MERGE builder runs with `execute()`.
- g1 = `whenMatchedUpdate`: `whenMatchedUpdate()` needs a `set=` mapping of columns. With no arguments you want `whenMatchedUpdateAll()`.
- g2 = `whenNotMatchedBySourceDelete`: `whenNotMatchedBySourceDelete()` deletes target rows that are missing from the source. With a one-wave source it would delete every other wave.

A3-L1 exercise 3

Hints. (1) Work through the first run: which clause fires for each row? (2) On the second run, does any source row lack a match? (3) Matched rows update in place. Updates do not add rows.

Answer.

```
3
```

Run one inserts 3 rows. Run two matches all 3 and updates them with identical values. The count stays 3. That is idempotence.

Common wrong answers.

- `6`: 6 is what an append would give. On the second run every source row matches an existing target row, so the MERGE updates instead of inserting.
- `0`: The first run inserts all 3 rows, because none of them matches an empty table.

A3-L1 exercise 4

Hints. (1) The grain is respondent per wave. Which columns identify one row? (2) Compare the deduplication keys with the MERGE condition. (3) A `W2` row for respondent R1 matches the `W1` row for R1, and overwrites it.

Answer. The bug is on line 6.

Fixed version:

```
1  def upsert_respondents(spark: SparkSession, source: DataFrame) -> None:
2      """MERGE one wave of respondents into silver. Grain: respondent per
       wave."""
3      latest = latest_per_key(source, ["respondent_id", "wave_id"],
       "delivery_version")
4      (
5          DeltaTable.forName(spark,
       "research_silver.nat_respondents").alias("t")
6          .merge(latest.alias("s"), "t.respondent_id = s.respondent_id AND
       t.wave_id = s.wave_id")
7          .whenMatchedUpdateAll()
8          .whenNotMatchedInsertAll()
9          .execute()
10     )
```

The condition matches on `respondent_id` alone. A wave `W2` row for a respondent matches that respondent's `W1` row and overwrites it, including `wave_id`. Match on the full grain: `t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id`.

Common wrong answers.

- Line 3: Line 3 deduplicates on both key columns, which matches the grain. Look at what the MERGE matches on.
- Line 7: `whenMatchedUpdateAll()` is right for a redelivery. The problem is which rows count as a match.

A3-L1 exercise 5

Hints. (1) You need a window, a rank and a filter. (2) `Window.partitionBy(*keys).orderBy(F.col(order_col).desc())` sorts each key's rows newest first. (3) Add `F.row_number().over(window)` as a column, keep rank 1, then drop the helper column.

Answer.

```
1  from pyspark.sql import DataFrame, Window
2  from pyspark.sql import functions as F
3
4
5  def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
       DataFrame:
```

```
    """Keep the row with the highest order_col for each key.

    Args:
        df: Source rows, possibly with repeated keys.
        keys: Columns that identify one target row.
        order_col: Column that ranks versions. Highest wins.

    Returns:
        One row per key.
    """
    window = Window.partitionBy(*keys).orderBy(F.col(order_col).desc())
    return (
        df.withColumn("_rank", F.row_number().over(window))
        .filter(F.col("_rank") == 1)
        .drop("_rank")
    )
```

A window partitioned by the keys and ordered by `order_col` descending puts the newest row first in each key. `row_number()` numbers them 1, 2, 3. Keep 1, drop the helper column.

A3-L1 exercise 6

Hints. (1) Nothing unchecked should reach silver. (2) The MERGE needs a source with one row per key. (3) The run log records the outcome of everything before it.

Answer.

1. Validate the bronze delivery against its data contract
2. Rename and cast columns from the Variable Registry
3. Keep the newest row per respondent and wave
4. MERGE into silver on respondent_id and wave_id
5. Run the table health check on silver
6. Write one record to the pipeline run log

Check the delivery, map it to canonical names, deduplicate, merge, confirm the table is healthy, then log the run with its final status.

Common wrong answers.

- Deduplicating after the MERGE is too late. The MERGE has already failed, or has already inserted the repeated rows.
- The contract describes the delivery as received. Check it before you rename, or the column names in the contract no longer match.
- Log last, so the run record includes the health check result and the final status.

A3-L2 Schema enforcement is on by default. Evolve on purpose.

A3-L2 exercise 1

Hints. (1) Is schema evolution on or off by default? (2) The table has no region column. (3) Enforcement rejects the write.

Answer. c) The append fails with a schema mismatch. Because: r1) Schema enforcement rejects writes with columns the table does not have.

Enforcement is on by default, so a write with an extra column fails. You opt in to evolution per write, or change the table with `ALTER TABLE`.

Common wrong answers.

- `a`: That needs `mergeSchema`. Without it, enforcement rejects the new column.
- `b`: An append does not drop columns. A MERGE with updateAll does, which makes it the dangerous case.
- `d`: A rescue column is an Auto Loader feature (`_rescued_data`), not a Delta write behaviour.
- `c|r2`: Right outcome, wrong reason. Delta schemas can change, with mergeSchema, withSchemaEvolution or ALTER TABLE. The default is to enforce.

A3-L2 exercise 2

Hints. (1) Does a MERGE fail on an extra source column the way an append does? (2) updateAll and insertAll work on the target's columns. (3) The extra column is ignored without an error.

Answer.

```
1 ['id', 'w']
```

Without schema evolution, `updateAll` and `insertAll` use the target's columns and quietly ignore `region`. Your own column check is the only guard here.

Common wrong answers.

- `['id', 'w', 'region']`: Only with `.withSchemaEvolution()` on the builder. Without it, updateAll and insertAll copy the target's columns and ignore region.
- `AnalysisException`: No error. The MERGE succeeds and the extra source column is silently ignored. That is why this case is dangerous.

A3-L2 exercise 3

Hints. (1) The builder method names what it enables. (2) It exists on DeltaMergeBuilder in delta-spark 4.0. (3) `withSchemaEvolution`.

Answer.

```
(
    DeltaTable.forName(spark, table).alias("t")
    .merge(source.alias("s"), condition)
    .withSchemaEvolution()
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)
```

`withSchemaEvolution()` turns evolution on for this MERGE only, which is safer than a session-wide setting.

Common wrong answers.

- g1 = `mergeSchema`: `mergeSchema` is a write option for appends and overwrites. The MERGE builder uses `withSchemaEvolution()`.
- g1 = `option`: The MERGE builder has no `option` method for this. Call `withSchemaEvolution()`.

A3-L2 exercise 4

Hints. (1) Two set differences, one in each direction. (2) Wrap each in `sorted()`. (3) `assert_no_drift` calls `schema_drift` and raises if either list is non-empty.

Answer.

```
def schema_drift(actual: list[str], expected: list[str]) -> dict[str,
    list[str]]:
    """Compare delivered columns with the registry.

    Args:
        actual: Columns in the delivery.
        expected: Canonical columns from the registry.

    Returns:
        Sorted unexpected and missing column names.
    """
    return {
        "unexpected": sorted(set(actual) - set(expected)),
        "missing": sorted(set(expected) - set(actual)),
    }


def assert_no_drift(actual: list[str], expected: list[str]) -> None:
```

```
    """Raise ValueError when the delivery's columns differ from the registry."""
    drift = schema_drift(actual, expected)
    if drift["unexpected"] or drift["missing"]:
        raise ValueError(f"schema drift: {drift}")
```

Checking both directions matters because Delta treats them differently: an extra column fails an append, but a missing column lands as NULL without an error.

A3-L2 exercise 5

Hints. (1) Which line changes a setting? (2) What does that setting do to every MERGE in the session? (3) Remove the conf.set. Evolve deliberately with withSchemaEvolution() when a change is approved.

Answer. The bug is on line 3.

Fixed version:

```
def upsert_silver(spark: SparkSession, source: DataFrame, table: str, keys: list[str]) -> None:
    """MERGE a deduplicated source into silver."""
    condition = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (
        DeltaTable.forName(spark, table).alias("t")
        .merge(source.alias("s"), condition)
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )
```

Line 3 turns on automatic schema evolution for the whole Spark session, so every MERGE adds any new source column. Remove it. When you approve a new column, use `ALTER TABLE ... ADD COLUMN` or `.withSchemaEvolution()` on that one MERGE.

Common wrong answers.

- Line 8: updateAll is fine. It only adds columns when evolution is switched on, and something switched it on.
- Line 4: The condition is correct. Look for a setting that changes how every MERGE treats new columns.

A3-L3 Time travel, RESTORE, OPTIMIZE and VACUUM

A3-L3 exercise 1

Hints. (1) The old version's files still exist. (2) Which command makes an old version current? (3) RESTORE TABLE ... TO VERSION AS OF n.

Answer. b) `RESTORE TABLE` to the version before the bad write. Because: r1) The earlier version's files are still on storage, and RESTORE makes that version current again.

RESTORE points the table back at the old version's files, as a new version. History is preserved, so you can inspect the bad write later.

Common wrong answers.

- `a`: It works, but it takes hours. RESTORE takes seconds while the old files still exist.
- `c`: VACUUM deletes old files, which are the ones you need to recover. It makes things worse.
- `d`: OPTIMIZE compacts files in the current version. It does not change the data.
- `b|r2`: Right action, wrong reason. RESTORE writes a new version. The bad write stays in history.

A3-L3 exercise 2

Hints. (1) The time travel clause has three words. (2) The same clause appears in the second statement. (3) `VERSION AS OF` and `RESTORE`.

Answer.

```
1 SELECT * FROM research_gold.nat_metrics VERSION AS OF 41;
2 RESTORE TABLE research_gold.nat_metrics TO VERSION AS OF 41;
```

`VERSION AS OF n` reads a past version. `RESTORE TABLE ... TO VERSION AS OF n` makes it current.

Common wrong answers.

- g1 = `AT VERSION`: The Delta syntax is `VERSION AS OF`.
- g2 = `ROLLBACK`: ROLLBACK undoes a transaction. Delta's command to make an old version current is RESTORE.

A3-L3 exercise 3

Hints. (1) Does RESTORE delete history? (2) Every commit adds one version. (3) The next version after 3.

Answer.

```
1 4
```

RESTORE is a commit like any other, so it becomes version 4 with the contents of version 1. Versions 2 and 3 remain in history.

Common wrong answers.

- `1`: RESTORE does not rewind the version counter. It writes a new version whose content matches version 1.
- `3`: Version 3 is the DELETE. Every write, RESTORE included, adds a new version.

A3-L3 exercise 4

Hints. (1) Which command deletes files? (2) What retention does it use? (3) Line 3. Run VACUUM with the default 7-day retention and drop line 2.

Answer. The bug is on line 3.

Fixed version:

```
1 OPTIMIZE research_silver.nat_respondents ZORDER BY (wave_id);
2 VACUUM research_silver.nat_respondents;
3 ANALYZE TABLE research_silver.nat_respondents COMPUTE STATISTICS;
```

`RETAIN 0 HOURS` deletes every file no longer in the current version, so no earlier version is readable. Line 2 disables the check that would have blocked it. Run VACUUM with default retention, or set the retention properties to the window you need.

Common wrong answers.

- Line 1: OPTIMIZE compacts files and keeps history. It does not block time travel.
- Line 2: Line 2 disables the guard, which is bad, but on its own it deletes nothing. The line that removes the files is the problem.

A3-L3 exercise 5

Hints. (1) Which property decides how long VACUUM keeps old files? (2) The log must cover the same window. (3) VACUUM still runs, with default arguments.

Answer.

- Set both retention properties to 30 days. From Databricks Runtime 18.0, the log retention must be at least the deleted-file retention.
- Keep running a plain `VACUUM t`. It respects the 30-day property and stops storage growing without limit.

Both retention properties at 30 days keep every version from the last month readable. A plain VACUUM then cleans up only what is older.

Common wrong answers.

- deleted = `d7`: With 7 days, VACUUM removes files after a week, so versions older than 7 days are unreadable. The policy needs 30.
- vacuum = `none`: Without VACUUM, storage grows with every write. Keep it, and let the retention property protect 30 days.
- vacuum = `zero`: RETAIN 0 HOURS overrides the property and deletes history immediately.
- log = `d7`: The log must keep entries at least as long as the data files, or the versions cannot be reconstructed.